

P++とは何だったのか

Bring peace to the galaxy with P++

[2019-08-28]

第141回 PHP勉強会@東京

#phpstudy

！ お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE
- GitHub/Packagistでは id: zonuexe
- ピクシブ株式会社 pixiv事業本部
- Emacs Lisper, PHPer
 - Emacs PHP Modeのメンテナ引き継ぎました
 - 好きなリスプはEmacs Lispです
- Qiitaに記事を書いたり変なコメントしてるよ

pixiv



Friends of Emacs-PHP development

Join us!

 <?php

- Repositories 25
- People 5
- Teams 1
- Projects 0
- Settings

Find a repository... Type: All Language: All

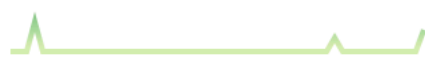
Customize pinned repositories  New

php-runtime.el

PHP-Emacs bridge, call PHP function from Emacs

- php
- emacs
- melpa

 Emacs Lisp ★ 2  1  GPL-3.0 Updated 2 days ago



phpactor.el

Interface to Phpactor (an intelligent code-completion and refactoring tool for PHP)

 Emacs Lisp ★ 8  4 1 issue needs help Updated 3 days ago



php-mode

A PHP mode for GNU Emacs



Top languages

 Emacs Lisp  PHP

Most used topics

Manage

emacs melpa php

major-mode

People 5 >



Invite someone



Search or jump to...



[Pull requests](#)

[Issues](#)

[Marketplace](#)

[Explore](#)



emacs-jp

📍 Japan

🔗 <http://emacs-jp.github.com/>

Repositories 18

People 37

Teams 8

Projects 0

Settings

Find a repository...

Type: All ▾

Language: All ▾

[Customize pinned repositories](#)

New

reading-init.el

init.el読書会のページ

[html](#)

[dotfiles](#)

[emacs](#)

[emacs-lisp](#)

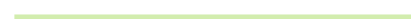
● Ruby ★ 6 🍴 1 Updated 6 days ago



helm-c-yasnippet

Helm source for yasnippet

● Emacs Lisp ★ 14 🍴 5 Updated on 21 Mar



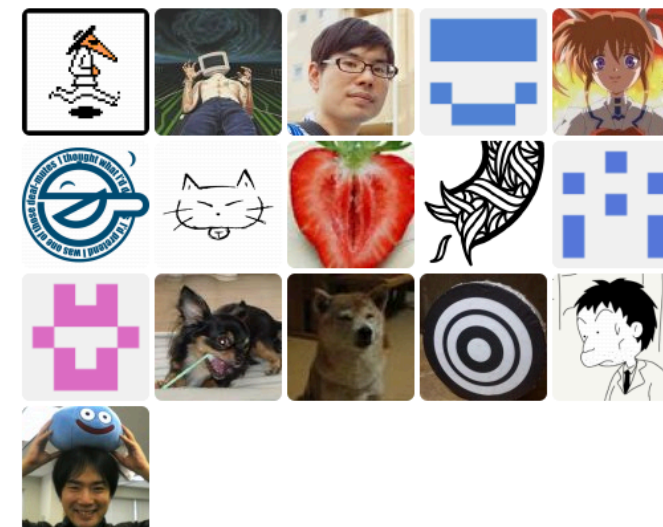
replace-colorthemes

Top languages

● Emacs Lisp ● Ruby ● TeX

People

37 >



P++とは何か

P++: 静的型付けをめざすPHP

PHP: [pplusplus:faq](#)

PHP 8から、PHPは「PHP」と「P++」という2つの言語を提供するようになる。P++はPHPとの下位互換性を削りながら除々にPHPを静的型付け言語にする試みだ。

PHP開発者の中には2つの流派がある。PHPの源流であり現在の形である動的型付け言語としてのPHPを良しとする流派と、PHPをより強い静的型付け言語へと発展させたい流派だ。良い悪いの問題ではない。どちらの流派も正当な理由がある。しかし、ゆるふわな動的型付け言語とガチガチの静的片付け言語は同じ一つの言語として同居できない。

そこで、コードネームP++として、PHPを静的型付け言語に発展させる新しい言語の開発が提案された。P++はforkではなく、PHPと同じコードベースを共有する。PHP 8のバイナリはPHPとP++を同時に実装する。言語の切り替えは何らかの宣言によって指定する。

P++はPHPの厳格な下位互換性を諦め、少しずつ下位互換性を切り捨てつつ、PHPを静的片付け言語にしていく。

これはPerl 5/6やPython 2/3という過去の失敗に学んだのだろう。

PHPとP++は同じコードベースであり、ほとんどのコードは共有するので、PHPの開発リソースが2倍必要になることはない。

PHP 7.4をLTSにしてPHP 8から介護感性を切り捨てるのは良いアイデアではない。言語を分断させてしまう。Python 2/3がすでに犯した失敗だ。

PHP 8はPHPとP++を含むので、PHP 8をインストールするとP++もついてくる。同じコードベースでPHPとP++を共存させることもできる。

PHPの開発が止まるわけではない。ただし静的型付け機能はPHPには入らずP++に入る。PHPは下位互換性を重視する。

言ってることは
正しい

情報は
間違ってる

どうい
うことか

さて

アンケート

君の想像するPHP
はどんなやつ？

| A

```
<!DOCTYPE html>
<html><head><title>Hello, PHP!</title></head>
<p>
  <?php if (date('H') < 19): ?>
    こんにちは
  <?php else: ?>
   こんばんは
  <?php endif; ?>
</p>
<p>今は<?= htmlspecialchars(date('H')) ?>時です</p>
```

I B

```
<?php require 'bootstrap.php';  
$db = mysqli_connect();  
$stmt = mysqli_prepare($db, 'SELECT * FROM books');  
mysqli_stmt_execute($stmt);  
print '<h1>本の一覧</h1><ul>';  
foreach ($stmt as $s) {  
    print '<li>'; print $s->name; print '</li>';  
}
```

I C

```
<?php
```

```
namespace Hoge\Http\Controller;
```

```
class BooksController extends BaseController {  
    public function index() {  
        $books = Books::getAll();  
  
        $this->render(compact("books"));  
    }  
}
```

A?

B?

C?

PHPはどんな
速度で変化してるか

速い？
遅い？

PHPは
変わったと思う？

速くなつたと
したらいつから？

これ何の数か
わかる？

PHP7.0... 46

PHP7.1... 28

PHP7.2... 22

PHP7.3... 13

PHP7.4... 26

お前はPHP7以降
追加された機能の
数を覚えているか

せいかい
バージョンごとに
受理されたRFC数

RFC?

Request for Comments

This page gives an overview of the current RFCs for PHP.

To create a new RFC, see [How To Create an RFC](#).

Note: An RFC is effectively “owned” by the person that created it. If you want to make changes, get permission from the creator. If no agreement can be found, the only course of action is to create a competing RFC. In this case, the old RFC page should be modified to become an intermediate page that points to all the competing RFC's.

A new page in this RFC namespace will automatically be loaded with an RFC [template](#). Customize as needed.

In voting phase

Under Discussion



PHP7の機能追加 や改廃の提案

Pythonでの
PEEPに相当

RFCの提案をもとに、
PHP開発者

(php.netのVCSアカウント所有者)が
一人一票を投じる仕組み

単純に
「追加された数」
ではないが

PHPの行く末を
知りたければRFCを
ウォッチすればよい

Request for Comments

This page gives an overview of the current RFCs for PHP.

To create a new RFC, see [How To Create an RFC](#).

Note: An RFC is effectively “owned” by the person that created it. If you want to make changes, get permission from the creator. If no agreement can be found, the only course of action is to create a competing RFC. In this case, the old RFC page should be modified to become an intermediate page that points to all the competing RFC's.

A new page in this RFC namespace will automatically be loaded with an RFC [template](#). Customize as needed.

In voting phase

Under Discussion



<https://wiki.php.net/rfc>

php RFC Watch

[Notify me when the vote for RFC is completed](#)

[array_key_first\(\), array_key_last\(\) and array_value_first\(\), array_value_last\(\)](#)

Add array_value_first() and array_value_last()?



Votes cast: 33 Yes: 15 (45%) No: 18 (55%)

Add array_key_first() and array_key_last()?



Votes cast: 32 Yes: 18 (56%) No: 14 (44%)

[Deprecations for PHP 7.3](#)

Deprecate (and subsequently remove) pdo_odbc.db2_instance_name php.ini directive?

[iterable to array\(\) and iterable count\(\)](#)

Add iterable_count()?



Votes cast: 37 Yes: 2 (5%) No: 35 (95%)

Add iterable_to_array()?



Votes cast: 32 Yes: 8 (25%) No: 24 (75%)

[User-defined object comparison](#)

Support user-defined object comparison?



Votes cast: 17 Yes: 5 (29%) No: 12 (71%)

<https://php-rfc-watch.beberlei.de/>

PHP RFC Bot



RFC Bot

ツイート
406フォロー
1フォロワー
1,020[フォローする](#)

PHP RFC Bot

@PHPRFCBot

Keep track of new and updated
[@php_net](#) RFCs. Managed by
[@michaeldyrynda](#). Also check out [php-rfc-watch.beberlei.de](#) for voting status.

📍 Adelaide, South Australia

📅 2016年5月に登録

Twitterを使ってみよう

登録してあなただけのタイムラインを作りましょう

[アカウント作成](#)

ツイート

ツイートと返信



PHP RFC Bot @PHPRFCBot · 8月2日

#PHP RFC moved to Implemented PHP 7.3: Same Site Cookie
[wiki.php.net/rfc/same-site-...](#)



10



16



PHP RFC Bot @PHPRFCBot · 7月21日

#PHP RFC moved to Implemented PHP 7.3: Deprecations for PHP 7.3
[wiki.php.net/rfc/deprecatio...](#)



14



9



PHP RFC Bot @PHPRFCBot · 7月18日

New #PHP Under Discussion RFC: Namespace Visiblity for Class, Interface and Trait
[wiki.php.net/rfc/namespace-...](#)



1



12



18



PHP RFC Bot @PHPRFCBot · 7月17日

#PHP RFC moved to Declined: iterable to `arrav()` and `iterable count()`

#externals

Opening PHP's #internals to the outside.

Search with  algolia

Status of ci.qa.php.net?

 30 days ago  Christoph M. Becker  6

object destruction php 5.6 vs. 7.2

 16 days ago  Nicolai Scheer  2

default_encoding RFC – aftermath

<https://wiki.php.net/rfc>

改めて数を見
てみよう

PHP7.0... 46

PHP7.1... 28

PHP7.2... 22

PHP7.3... 13

PHP7.4... 26

意外に変わってる
なっで感じはする

とてもざっくり言うとしガ
シー仕様は消え、不条理な
挙動は是正され、他言語に
あるようなカッコイイ機能
が追加提案されてる



ところで
Rasmusは
こう言った

*“We have things like protected properties.
We have abstract methods. We have all
this stuff that your computer science
teacher told you you should be using.
I don't care about this crap at all.”*

–Rasmus Lerdorf

https://en.wikiquote.org/wiki/Rasmus_Lerdorf

“PHPにはprotectedプロパティも抽象メソッドもありますよ。計算機科学の教授が「使え」と言ってるものは全部。そんなことは~~クソ~~興味ないですけど”

—Rasmus Lerdorf

改めてPHPの言語
仕様が追加された時
系列を見てみよう

| PHP 7.4

- Arrow functions 2.0
 - JavaScriptなど
- Typed Property
 - Javaなどの静的型付き言語
- Numeric Literal Separator
 - Ada, Ruby, JavaScriptなど

| PHP 7.3

- Flexible Heredoc and Nowdoc
 - Rubyにも似た記法はある
- list() Reference Assignment
 - これはPHP独自に近い

| PHP 7.2

- Trailing Commas In List Syntax
 - カンマ(,)を並べて書く記法のほぼ全てで余分につけてもよくなった
- Parameter Type Widening
 - 継承したクラスで引数の型省略

| PHP 7.1

- Nullable Types
- Void Return Type
- Multi catch
- Allow specifying keys in list()

| PHP 7.0

- Anonymous Class
- Generator Delegation(yield from)
- Return Type Declaration
- Null Coalesce Operator
- Scalar type hints

| PHP 5.6

- Exponential Operator(pow**)
- Importing named function
- Variadic function argument (...)
 - 配列を引数として展開
- Argument unpacking(...)
 - 可変長引数を配列として取得

| PHP 5.5

- Generator
- finally keyword (try-catch-...)
- Const array/string dereference
- Allow non-scalar keys in foreach

| PHP 5.4

- Traits
- Short array syntax
- callable type hint

| PHP 5.3

- Closures (function expression)
 - 関数式、クロージャ、またはラムダ式
 - JavaScriptのfunctionに似てるが、レキシカルスコープ(変数の捕捉)のためにuseキーワードが必要

抽象クラス, トレイト, インター
フェイス, ジェネレータ, イテレー
タ, クロージャ, 関数の動的呼び出
し, 無名クラス, eval, 型宣言, 連
想配列, リフレクションAPI, 自己
文書化, 標準入出力の読み書き,
REPL, クラスの遅延ロード

これらの新機能をフルに
活用してコードを書いて
る人はどれだけ居るの？

(そういう人は居るけど、
そうじゃない人たちと二
極化してるのは確実)

PHPの古参開発
者たちは冷ややか

PHP以外の言語に
触れてた経験のない
みんなもポカーソ

そういうところやぞ

クールな新機能や
静的解析しやすい
仕様に興味あるひと

従来のコードが動けば十分で破壊的進化は望まないひと

そのほかにもPHP
ユーザには様々な立
場のひとたちが居る

その見えない溝が
可視化されたのが
今回の一件

さらにひきがねを
引く事件があった

◀ [RFC] [VOTE] Deprecate PHP's short open tags, again

✦ Posted [21 days ago](#) by **G. P. B.** — [view source](#)



The voting for the "Deprecate short open tags, again" [1] RFC has begun. It is expected to last two (2) weeks until 2019-08-20.

A counter argument to this RFC is available at https://wiki.php.net/rfc/counterargument/deprecate_php_short_tags

Best regards

George P. Banyard

[1] https://wiki.php.net/rfc/deprecate_php_short_tags_v2

✦ Posted [21 days ago](#) by **Chase Peeler** — [view source](#)



I'm not a voter, but, I have a question. If this fails, does that mean the original RFC that passed is still in effect?

If I did have a vote, I would be against this RFC for the reasons laid out by Zeev in the counter argument. However, I feel the negative impacts of possible code leaks that will be caused by the original RFC are even worse. If I did have a vote, I might feel forced to vote for this. At least with this RFC we wouldn't really see any of the other negative impacts until PHP 9, given us time to revisit and possibly reverse that part of this RFC.

The voting for the "Deprecate short open tags, again" [1] RFC has begun. It is expected to last two (2) weeks until 2019-08-20.

A counter argument to this RFC is available at https://wiki.php.net/rfc/counterargument/deprecate_php_short_tags

Best regards

George P. Banyard

[1] https://wiki.php.net/rfc/deprecate_php_short_tags_v2

| Deprecate PHP's short open tags, again

- 一般的なPHPタグは<?php ... ?> または<?= ... ?> だが、<? ... ?>という形式もある
- php.iniで有効無効を切替られる
- 環境によってOnだったりOffだったり

| Deprecate PHP's short open tags, again

- XML宣言との相性がとても悪い
- そんな半端モンは引導を渡してやろっぜ！ ということで提案された
- 構文がシンプルになるし、機械的に変換もできるから良いだろ、という主旨

Zeevによる 反論

Counterargument to Deprecate Short Tags RFC V2

- Date: 2019-08-06
- Author: Zeev Suraski, zeev@php.net

Introduction

An proposal has been made to deprecate PHP's short tags syntax. The proposal was first made available in [V1 here](#), and a newer version was later made available in [V2 here](#). The goal of this page is to provide a counterargument for this proposal, or in other words, to explain why we should not be deprecating this syntax.

Overview

The motivation for removing short tags as it is illustrated in the V2 RFC is quite weak. Essentially - it does not present *any* new motivations that did not exist 20+ years ago, when this syntax was first introduced. At the time, all the different potential opening and closing tags were heavily discussed, and all the implications - including the ones brought up by the deprecation RFC - were already known. There is no new 'evidence' available here.

– Table of Contents

Counterargument to Deprecate Short Tags RFC V2
Introduction
Overview
Analyzing The Motivations For Deprecation
The Case For Keeping Short Tags
Additional Thoughts
Summary

| Zeev Suraski

- PHP3・4の開発者
 - 相方のAndi Gutmansとともに
VM型処理系のZend Engine開発
- PHPでよくあるZend…は
ZeevとAndiに由来
- PHPコードに未だに影響を残す

| Rasmus Lardorf

- オリジナルのPHP(PHP/FI)作者
- 言語の原作者は最終的な決定権を担っていることがある(BDFL:優しい終身の独裁者)が、Rasmusはそうではない
- 必要な場面では発言するが権限はRFCの一票しか持ってない
- 今回の一件にはあまり関与してない

| Nikita Popov

- 近年のPHPの中核的な開発者
- PHP5.5以降、非常に多数の言語機能を実装してPHPに取り入れてきた
- PHP-Parserなどの作者
- 去年の大学院を卒業し今年JetBrainsに入社してPhpStormチームに参加





燃えた

めちやくちや
燃えた

| Deprecate PHP's short open tags, again

- <? ... ?> 使っていないひとは使っていないし、廃止したからと言って文法が簡単になるか…？
- この仕様を削除してもパフォーマンスは特に向上しないし、コードもちょっとしか消えない

Deprecate PHP's short open tags, again

- 大山鳴動して鼠一匹
- 大騒ぎして文法変更する割に得られるメリットが少なすぎないか…？
- だからと言って負債になった文法はずっと変えられないままなのか？！
- 喧喧諤諤非難轟々

再び立ち上がった

◀ Bringing Peace to the Galaxy

✈ Posted [19 days ago](#) by **Zeev Suraski** — [view source](#)



[... and not in the Sith Lord kind of way.]

Looking at some of the recent (& not so recent) discussions on internals@, some of the recent proposals, as well as some of the statements made regarding the future direction of the language - makes it fairly clear that we have a growing sense of polarization.

As Peter put it yesterday (I may be paraphrasing a bit) - some folks just want to clear some legacy stuff. I think that in practice it goes well beyond that - many on internals@ see parts of PHP as in bad need of repair (scoop: I agree with some of that), while other capabilities, that exist in other competing languages - are - in their opinion - sorely missing.

At the other end of the spectrum, we have folks who think that we should retain the strong bias for downwards compatibility we always had, that PHP isn't in dire need of an overhauling repair and that as far as features go

- less is more - and we don't have to race to replicate features from other languages - but rather opt for keeping PHP simple.

To a large degree, these views are diametrically opposed. This made many internals@ discussions turn into literally zero sum games - where when one side 'wins', the other side 'loses', and vice versa.

PHPコミュニティ
内の革新vs保守の
対立を解決したい

ここで提案
されたのがP++

概要は最初の
本の虫の記事で
紹介された通り

P++: 静的型付けをめざすPHP

PHP: [pplusplus:faq](#)

PHP 8から、PHPは「PHP」と「P++」という2つの言語を提供するようになる。P++はPHPとの下位互換性を削りながら除々にPHPを静的型付け言語にする試みだ。

PHP開発者の中には2つの流派がある。PHPの源流であり現在の形である動的型付け言語としてのPHPを良しとする流派と、PHPをより強い静的型付け言語へと発展させたい流派だ。良い悪いの問題ではない。どちらの流派も正当な理由がある。しかし、ゆるふわな動的型付け言語とガチガチの静的片付け言語は同じ一つの言語として同居できない。

そこで、コードネームP++として、PHPを静的型付け言語に発展させる新しい言語の開発が提案された。P++はforkではなく、PHPと同じコードベースを共有する。PHP 8のバイナリはPHPとP++を同時に実装する。言語の切り替えは何らかの宣言によって指定する。

P++はPHPの厳格な下位互換性を諦め、少しずつ下位互換性を切り捨てつつ、PHPを静的片付け言語にしていく。

これはPerl 5/6やPython 2/3という過去の失敗に学んだのだろう。

PHPとP++は同じコードベースであり、ほとんどのコードは共有するので、PHPの開発リソースが2倍必要になることはない。

PHP 7.4をLTSにしてPHP 8から介護感性を切り捨てるのは良いアイデアではない。言語を分断させてしまう。Python 2/3がすでに犯した失敗だ。

PHP 8はPHPとP++を含むので、PHP 8をインストールするとP++もついてくる。同じコードベースでPHPとP++を共存させることもできる。

PHPの開発が止まるわけではない。ただし静的型付け機能はPHPには入らずP++に入る。PHPは下位互換性を重視する。

誰も何も決定して
ないことを除けば

PHPの意思決定機関が
開発者によるRFCの多
数決であることを知って
いれば誤解はしないはず

本の虫で紹介された
「ソース」はメーリン
グリストでの議論の
ためのメモ書き

燃え広がる
誤解...



491



3



@tadsan 2019年08月15日に更新 30904 views

編集する

P++: 銀河に平和をもたらすための奇策と決着

PHP

p++

PHP 8から、PHPは「PHP」と「P++」という2つの言語を提供するようになる

というキャッチーな紹介をするP++: 静的型付けをめざすPHPという記事がそれなりに話題になり、このニュースは目覚しく革新的な内容で、多くのひとの目を引き付けました。

これは早まった理解であり、ほとんど誤報と言ってもいい内容でした。2019年8月15日には提案者本人も、少なくとも「P++」の計画を短期的に実現するとは非現実的であり時期尚早であることを認めています。

この記事では、PHP開発の現状、なぜ野心的なP++計画が提案され、事実上撤回されたかの経緯について紹介します。

その後の経緯
はこの記事に
書いた通り

[start](#) › [rfc](#) › [p-plus-plus](#)

Poll: Feasibility of P++

- Version: 0.1
- Date: 2019-08-14
- Author: Derick, derick@php.net
- First Published at: <http://wiki.php.net/rfc/p-plus-plus>

Introduction

There are continuous arguments whether the PHP project should do everything to preserve backwards compatibility, or continue to add features to make PHP a more modern language, with as an example enhanced stricter type-safety.

Several suggestions have been made to address this perceived split. One of them is creating a newly branded “P++” to put all the new features into, and perhaps go the more “strict way”, while continuing to maintain “PHP” without making this original version change at all. As it takes time, attention, and resources, to see whether such a direction is feasible, it makes sense to find out whether the general internals community thinks it is a worthwhile endeavour.



理想はわかるが…
現実的ではない…

Zeevの予言……



Hi,

In the last week(s) there has been a lot of chat about Zeev's P++ idea. Before we end up spending this project's time and energy to explore this idea further, I thought it'd be wise to see if there is enough animo for this. Hence, I've created a document in the wiki as a poll:

All,

Using a humoristic tone, I'm happy that finally internals@ is so unified. I almost get the feeling that you may not like the idea...

On a more serious note, I'll keep the feedback on the validity of this vote in just about every aspect (process, jurisdiction, anything really) to myself, and say just two things:

- The P++ idea makes absolutely no sense in vacuum. The reception around this idea implied a decision between 'one big happy family' and 'a split'. Since at this stage these are the perceived choices - I'd vote against it too (which I just did, why not). However, I believe it's a false choice.
- It will absolutely make sense to discuss it when it'll start becoming clearer to everyone that 'one big happy family' is really not an option. We'd be choosing how to soft split the family - granularly (2^n dialects), into many editions (n dialects), or into two separate dialects with clearer mandates (2 dialects). I get it that it's intangible for many of us (myself included, to a degree), which is why this idea is perceived as the 'evil splitter' for everyone to unite and rally against. Maybe I'm wrong,

PHPの明日
はどっちだ

救いとしてはそれ以降も
別のRFCで建設的な議
論は続いてるし、Zeev
もNikitaも参加している

PHP先生の
次回作(PHP8)に
ご期待ください！